

TéléPro CI

Analyse — Gestion Multi-Magasin

Document d'analyse technique et fonctionnelle

Avril 2026

Ce document analyse les impacts techniques et fonctionnels de l'ajout de la gestion multi-magasin dans l'application TéléPro CI. Il couvre les modifications de schéma, les changements d'API, les évolutions frontend et la stratégie de migration recommandée.

Résumé exécutif

La gestion multi-magasin est une extension structurelle qui touche l'ensemble des couches de l'application. Le périmètre principal est la notion de stock par magasin (remplacement de produits.stock par une table de liaison) et le filtrage automatique de toutes les données selon le magasin de l'utilisateur connecté.

Effort estimé par domaine

Domaine	Complexité	Raison principale
Schéma DB	& Élevée	Stock par magasin = réécriture table produits
API — routes/index.ts	& Élevée	~40 queries à filtrer + transactions stock
API — routes/ventes.ts	%È Moyenne	Triggers + INSERT stock à adapter
Auth / JWT	%È Moyenne	magasin_id dans payload + middleware
Frontend	%Ç Faible	Sélecteur magasin + filtres UI
Rapports	%È Moyenne	Vue par magasin + vue consolidée
Migration données	& Élevée	Backfill stock existant sur un magasin

1 · Modifications du schéma PostgreSQL

1.1 Nouvelle table magasins

```
CREATE TABLE magasins (  
  id          SERIAL PRIMARY KEY,  
  code        VARCHAR(20)  UNIQUE NOT NULL,  
  nom         VARCHAR(100) NOT NULL,  
  adresse     TEXT,  
  telephone   VARCHAR(20),  
  email       VARCHAR(100),  
  actif       BOOLEAN DEFAULT TRUE,  
  created_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

1.2 Stock par magasin (changement majeur)

La colonne produits.stock actuelle devient obsolète. Elle est remplacée par une table de liaison qui porte le stock disponible par produit et par magasin.

```
-- Nouvelle table (remplace produits.stock)  
CREATE TABLE stocks (  
  produit_id  INTEGER REFERENCES produits(id) ON DELETE CASCADE,  
  magasin_id  INTEGER REFERENCES magasins(id) ON DELETE CASCADE,  
  quantite    INTEGER NOT NULL DEFAULT 0,  
  stock_alerte INTEGER NOT NULL DEFAULT 5,  
  PRIMARY KEY (produit_id, magasin_id)  
);  
  
-- Backfill : créer une entrée pour le magasin principal (id=1)  
INSERT INTO stocks (produit_id, magasin_id, quantite, stock_alerte)  
SELECT id, 1, stock, stock_alerte FROM produits;  
  
-- Puis supprimer les colonnes dépréciées  
ALTER TABLE produits DROP COLUMN stock;  
ALTER TABLE produits DROP COLUMN stock_alerte;
```

1.3 Colonnes magasin_id à ajouter

Table	Type de colonne	Impact
utilisateurs	magasin_id INTEGER (NULL = accès tous)	Scoping auth
ventes	magasin_id INTEGER NOT NULL	Filtrage des ventes
achats	magasin_id INTEGER NOT NULL	Filtrage des achats
paiements	magasin_id INTEGER NOT NULL	Rapports caisse
mouvements_stock	magasin_id INTEGER NOT NULL	Traçabilité stock
depenses	magasin_id INTEGER NOT NULL	Comptabilité locale
devis	magasin_id INTEGER NOT NULL	Devis locaux
notifications	magasin_id INTEGER	Alertes ciblées

1.4 Transferts inter-magasins

```
CREATE TABLE transferts_stock (  
  id          SERIAL PRIMARY KEY,  
  produit_id  INTEGER REFERENCES produits(id),  
  magasin_source INTEGER REFERENCES magasins(id),  
  magasin_dest  INTEGER REFERENCES magasins(id),
```

1.5 Vues à adapter

- v_creances_clients !' ajouter magasin_id dans le SELECT et créer une vue consolidée
- v_dettes_fournisseurs !' idem
- v_echeances_30j !' paramétrer par magasin ou créer une vue globale séparée
- Triggers trg_paiement_vente / trg_paiement_achat !' inchangés (portent déjà sur la ligne)

2 · Modifications API (Node.js / Express)

2.1 JWT — ajout de magasin_id dans le payload

Le token JWT embarque déjà : sub, email, nom, prenom, role, perms, jti. Il faut y ajouter magasin_id (null pour les admins multi-magasins).

```
// routes/auth.ts — construction du payload
const payload = {
  sub:      user.id,
  email:    user.email,
  nom:      user.nom,
  prenom:   user.prenom,
  role:     user.role_code,
  perms:    user.permissions,
  magasin_id: user.magasin_id, // !• NOUVEAU (null = tous)
};
```

2.2 Middleware requirePerm — scoping automatique

```
// middleware/auth.ts
// req.user.magasin_id disponible dans toutes les routes
// Les admins (magasin_id = null) voient tout

export function scopeMagasin(req: Request): number | null {
  return req.user?.magasin_id ?? null;
}
```

2.3 Pattern de filtrage dans les routes

Quasi toutes les queries dans routes/index.ts et routes/ventes.ts doivent appliquer le filtre magasin. Voici le pattern à suivre :

```
// Avant (mono-magasin)
const { rows } = await db.query(
  `SELECT * FROM ventes WHERE client_id = $1`, [client_id]
);

// Après (multi-magasin)
const magasin_id = req.user!.magasin_id;
const params: unknown[] = [client_id];
let sql = `SELECT * FROM ventes WHERE client_id = $1`;
if (magasin_id) { params.push(magasin_id); sql += ` AND magasin_id = ${params.length}`; }
const { rows } = await db.query(sql, params);
```

2.4 routes/ventes.ts — POST /api/ventes

La création d'une vente doit :

- Insérer magasin_id dans la table ventes
- Décrémenter le stock dans la table stocks (produit_id, magasin_id) et non plus produits.stock
- Vérifier l'alerte stock dans stocks, pas dans produits

```
// Avant
await client.query(
  `UPDATE produits SET stock=stock-$1 WHERE id=$2`,
  [l.quantite, l.produit_id]
);

// Après
await client.query(
  `UPDATE stocks SET quantite=quantite-$1
  WHERE produit_id=$2 AND magasin_id=$3`
```

2.5 Nouveaux endpoints nécessaires

Méthode + Route	Rôle	Permission requise
GET /api/magasins	Liste des magasins	admin
POST /api/magasins	Créer un magasin	admin
PUT /api/magasins/:id	Modifier un magasin	admin
GET /api/stocks?magasin_id=	Stock par magasin	stock
POST /api/stocks/transfert	Transfert inter-magasins	stock
GET /api/dashboard?magasin_id=	Dashboard par magasin	tous
GET /api/rapports/consolide	Rapport tous magasins	admin

3 · Modifications Frontend (React / Vite)

3.1 Sélecteur de magasin

Pour les utilisateurs admin (`magasin_id = null` dans le JWT), un sélecteur de magasin apparaît dans `AppLayout`. Le magasin actif est stocké dans un contexte React global et injecté dans tous les appels API.

```
// hooks/useMagasin.ts
export function useMagasin() {
  const { user } = useAuth();
  const [magasinActif, setMagasinActif] = useState(user.magasin_id);
  // Si user.magasin_id est non-null, le magasin est fixe (non commutable)
  return { magasinActif, setMagasinActif, isAdmin: !user.magasin_id };
}
```

3.2 Pages impactées

Page	Modification	Priorité
DashboardPage	KPIs filtrés par magasin + vue consolidée	Haute
StockPage	Afficher <code>stocks.quantite</code> par magasin	Haute
ProduitsPage	Stock consolidé tous magasins en lecture	Haute
VentesPage	Filtre magasin + colonne magasin dans tableau	Haute
AchatsPage	Idem ventes	Haute
CreancesPage	Filtre par magasin	Moyenne
RapportsPage	Onglets : par magasin / consolidé	Moyenne
AdminPage	CRUD magasins + affectation utilisateurs	Haute
AdminReferentielsPage	Référentiels globaux ou par magasin (à décider)	Faible

3.3 API client — injection automatique du filtre

```
// lib/api.ts
function buildParams(extra: Record<string, unknown> = {}) {
  const magasin_id = getMagasinActif(); // depuis contexte React
  return magasin_id ? { magasin_id, ...extra } : extra;
}

// Exemple d'usage
const ventes = await api.get('/ventes', { params: buildParams({ statut }) });
```

4 · Stratégie de migration recommandée

Phase 1 — Fondations DB (sans casser l'existant)

- Créer la table `magasins` avec un enregistrement initial « Magasin Principal »
- Créer la table `stocks` et backfiller depuis `produits.stock`
- Ajouter les colonnes `magasin_id` (NULLABLE avec DEFAULT 1) sur toutes les tables
- NE PAS encore supprimer `produits.stock` — double write temporaire

Phase 2 — API

- Ajouter `magasin_id` dans le JWT (non breaking si nullable)

- Mettre à jour le middleware pour exposer req.user.magasin_id
- Adapter les routes une par une avec le pattern de filtrage
- Créer les nouveaux endpoints /api/magasins et /api/stocks/transfert

Phase 3 — Frontend

- Ajouter le contexte MagasinProvider dans App.tsx
- Intégrer le sélecteur de magasin dans AppLayout
- Mettre à jour StockPage, DashboardPage, VentesPage en priorité
- Adapter RapportsPage pour la vue consolidée

Phase 4 — Finalisation

- Passer magasin_id NOT NULL sur toutes les tables (après vérification données)
- Supprimer produits.stock et produits.stock_alerte
- Mettre à jour les triggers et vues
- Tests d'intégration end-to-end

5 · Risques et points de vigilance

Risque critique — Triggers de stock

Les triggers `trg_paiement_vente` et `trg_paiement_achat` ne touchent pas au stock directement, mais toutes les routes qui font `UPDATE produits SET stock=...` doivent migrer vers `UPDATE stocks SET quantite=...`. Un oubli = stock global incohérent.

Risque élevé — Backfill des données existantes

Toutes les ventes, achats, paiements et mouvements_stock existants doivent recevoir `magasin_id=1` (magasin principal). À faire en migration SQL avant de rendre la colonne NOT NULL.

Risque élevé — Rapports consolidés

Les agrégats du dashboard (chiffre d'affaires, créances, dettes) doivent distinguer clairement le périmètre : un magasin seul vs. tous magasins. Un admin qui voit le CA d'un seul magasin alors qu'il croit voir le total = erreur de pilotage grave.

Point de vigilance — Référentiels

Les 13 tables référentiels (types clients, statuts, moyens paiement...) sont actuellement globales. Décider si elles restent globales (recommandé, plus simple) ou deviennent par-magasin (plus flexible mais complexe).

Point de vigilance — Numérotation des documents

Les numéros de vente (VTE-XXXX), bons de commande et devis sont séquentiels globalement. En multi-magasin, préférer un préfixe magasin : VTE-MG1-1001, VTE-MG2-1001. Modifier la logique de génération du numéro dans `routes/ventes.ts`.

6 · Récapitulatif des fichiers à modifier

Fichier	Type de changement	Effort
<code>migrations/013_magasins.sql</code>	Nouveau — table magasins + stocks	& & &
<code>migrations/014_backfill.sql</code>	Nouveau — backfill <code>magasin_id=1</code>	& &
<code>apps/api/src/lib/db.ts</code>	Inchangé	—
<code>apps/api/src/routes/auth.ts</code>	<code>magasin_id</code> dans JWT payload	&
<code>apps/api/src/routes/ventes.ts</code>	Stock stocks + <code>magasin_id</code> insert	& & &
<code>apps/api/src/routes/index.ts</code>	Filtrage + nouveaux endpoints	& & &
<code>apps/api/src/middleware/auth.ts</code>	Exposer <code>magasin_id</code>	&
<code>apps/api/src/services/notifications.ts</code>	<code>alerteStock</code> !' table stocks	& &
<code>apps/web/src/components/layout/AppLayout.tsx</code>	Sélecteur magasin	& &
<code>apps/web/src/pages/StockPage.tsx</code>	Refonte affichage stock	& & &
<code>apps/web/src/pages/DashboardPage.tsx</code>	KPIs par magasin	& &
<code>apps/web/src/pages/VentesPage.tsx</code>	Colonne + filtre magasin	& &
<code>apps/web/src/pages/AdminPage.tsx</code>	CRUD magasins	& &
<code>apps/web/src/pages/RapportsPage.tsx</code>	Vue consolidée	& &
<code>packages/shared/src/index.ts</code>	Types Magasin, Stock, Transfert	&

